Sujet de TP 1 : Problème MNIST

Le jeu de données **MNIST** est un benchmark classique en apprentissage automatique pour la classification d'images. Il contient des images en niveaux de gris de chiffres manuscrits (0 à 9). Chaque image est de taille **28×28 pixels**, soit 784 pixels au total. L'objectif est de prédire correctement la classe (le chiffre) associée à chaque image parmi les **10 classes** possibles (0–9). La figure 1 illustre ce jeu de données.

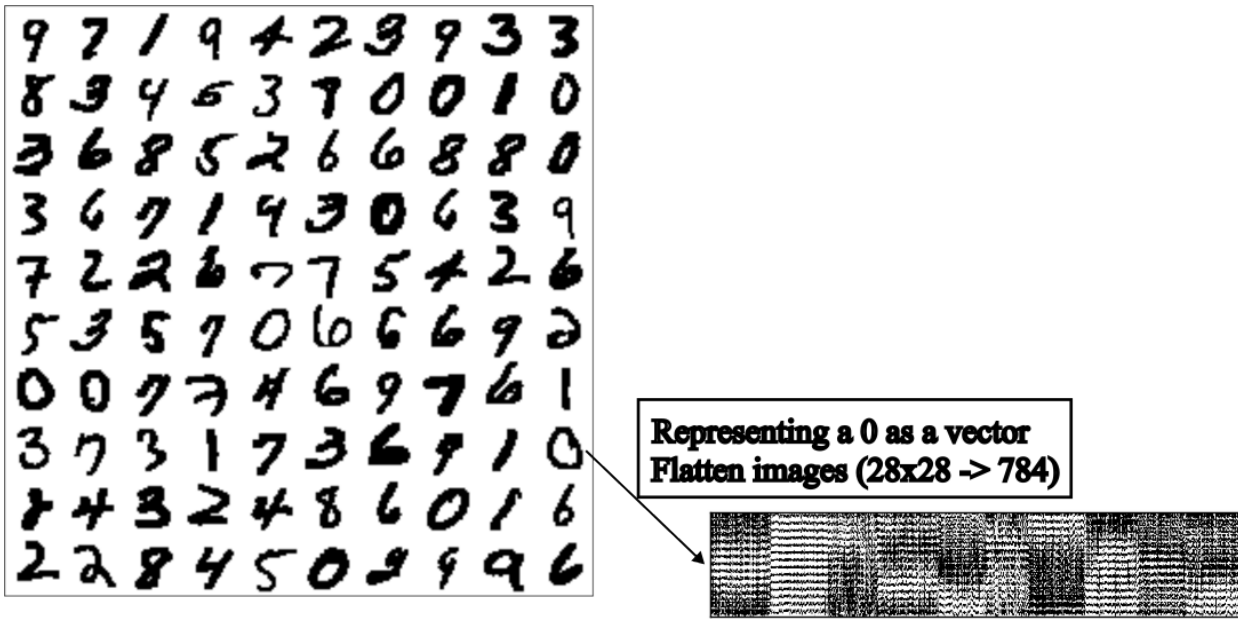

Figure 1. Dataset du problème MNIST et représentation vectorielle des 784 pixels.

La nomenclature utilisée est souvent w_i^j, où w est un vecteur ; i est l'indice du vecteur ; $j$est la classe que w représente. Enfin, West l'ensemble des vecteurs $[w^0, \dots, w^9]$, correspondant aux classes disponibles.

Pour entraîner un réseau de neurones avec la base MNIST, chaque image $[28 \times 28]$est transformée en un vecteur aplati $[1 \times 784]$. Ainsi, la position spatiale des pixels n'est plus explicitement exploitée dans le modèle. On propose d'implémenter un modèle de réseau des neurones denses, aussi connu comme *multi-layer perceptron* (MPL) ou *fully connected*. Il doit être composé :

- d'une couche d'entrée de dimension 784,
- d'une couche de sortie dense de 10 neurones, produisant une distribution de probabilité sur les 10 classes,
- d'un nombre des couches cachées différentes selon le besoin.

Chaque sortie représente la probabilité que l'image appartienne à une classe donnée. L'apprentissage pourra être réalisé par descente de gradient en minimisant l'entropie croisée (*cross-entropy loss*).

1. Choix de la Fonction d'Activation

Assumer l'optimiseur **adam** et la fonction coût **categorical_crossentropy** pour l'étude de la fonction d'activation.

a. Implémenter ce réseau des neurones avec deux couches (entrée et sortie) à partir de la fonction d'activation **softmax**
b. Implémenter ce réseau des neurones avec 2 ou 3 couches cachées et **ReLU** comme fonction d'activation.
c. Implémenter ce réseau des neurones avec 1 ou 2 couches cachées et **tanh** comme fonction d'activation.
d. Implémenter ce réseau des neurones avec 1 ou 2 couches cachées et **sigmoid** comme fonction d'activation.
e. Comparer les différentes implémentations de MPL selon l'architecture et fonction d'activation. Justifier les compromis entre : **accuracy**, nombre d'**epochs**, nombre de synapses (dimension de W normalement stocké en mémoire).

2. Choix de l'Algorithme d'Optimisation

Choisir la meilleure architecture de réseau des neurones dans l'ensemble des critères présentés pour faire varier les implémentations précédentes selon les optimiseurs suivants :

Optimizer	Compromis des Algorithmes
SGD	Base simple, classique, intuition sur descente de gradient
Adam	Moderne, converge vite, combine momentum + RMSprop
RMSprop	Adaptation du taux d'apprentissage, stabilité sur entrées variées
Adagrad	Learning rate par paramètre, illustration de l'effet adaptatif

3. Choix de la Fonction Coût

Le choix de la fonction de coût dépend du type de problème et de la sortie du modèle. Ajuster le modèle de manière conséquente pour pouvoir varier les implémentations précédentes selon les fonctions suivants :

Type de Problème	**Sortie du Modèle**	**Meilleure Fonction Coût**
Binary Classification (ex. spam / non spam)	Probabilité $0 \leq y \leq 1$	binary_crossentropy
Multi-Class Classification (ex. chien / chat / lapin)	Vecteur des probabilités de chaque classe	categorical_crossentropy *À utiliser si les labels sont encodés en one-hot (ex. [0,0,1,0,...])*
Multi-Class (One Label at a Time)	Logits (raw scores)	sparse_categorical_crossentropy *À utiliser si les labels sont des entiers (ex. 0,1,2,...). Pratique pour MNIST si les chiffres sont codés 0–9*